

Setonix to CANFAR uploader: workflow, integrity and operational results

Date: 8 October 2026 (Australia/Perth)

Script: `vcp_scratch_v3tk_v768_to_canfar.sh`

Local source: `/Users/Igniz/Desktop/ICRAR/vcp_scratch_v3tk_v768_to_canfar.sh`

1. Purpose and current result

The uploader copies selected nGIST v7.6.8 products from Setonix scratch to CANFAR ARC storage. Its main purpose is to make the destination files match the Setonix source files while avoiding unnecessary transfers. It checks existing files, repairs mismatches, uploads missing files and verifies replacements automatically.

The current implementation is one self-contained Bash script with embedded Python. It runs from Setonix. It does not require SSH into CANFAR, a manually running checksum watcher, a separate uploaded Python helper, or FITS downloads back to Setonix.

These integrity guarantees describe the default checksum-enabled mode. An explicit `--force-overwrite --no-checksum` mode is now available when checksum jobs cannot start. It uploads every selected available product after preserving existing destinations, but reports destination bytes as **UNVERIFIED**. Section 14 describes the command, safeguards and limits. The usage/help function is now immediately after the shell setup at the beginning of the script.

The supplied production log records a successful run over nine galaxies:

- **90/90 selected files verified:** ten products for each galaxy.
- **Five continuum cubes replaced:** their byte counts matched the sources, but their SHA256 hashes differed.
- **85 products already matched:** their science-file payloads did not need uploading.
- **Total elapsed time: 550 seconds, or 9 minutes 10 seconds.**
- The valid CADC certificate was reused; the shared read-only overlay avoided worker image copies.
- No reported transfer failures or outstanding verification remained at the end.

These results establish agreement with the Setonix files at the time of verification. They do not by themselves establish that every FITS file is scientifically correct or that the mass, SFR and EWHa pipelines have completed successfully.

2. Why the uploader changed

Earlier logs showed `vcp` reporting that the source and destination were identical and skipping the NGC4698 continuum cube, followed by unsuccessful read-back attempts. Repeating the same operation did not reliably repair the input needed by the downstream pipeline.

The essential change is to compare **hashes computed from actual file bytes on both systems**. The uploader no longer treats an existing remote object's size or a transfer tool's identical-file decision as sufficient evidence of integrity.

This distinction matters because files can have the same length and different contents. The latest production log demonstrates that directly: all five replaced continuum cubes had equal Setonix/CANFAR byte counts before repair, but different SHA256 hashes.

The logs establish the mismatches and successful replacements. They do not identify the physical origin of the differing bytes. A mismatch could represent corruption or a different product version; the script treats Setonix as the authoritative source and makes CANFAR match it.

3. Inputs, destinations and product selection

3.1 Run roots

RUN ARGUMENT	SETONIX SOURCE ROOT	CANFAR DESTINATION ROOT
normal	/scratch/pawsey1308/mauve/products/v3tk_v7.6.8	arc:projects/mauve/products/v3tk_v7.6.8
7000	/scratch/pawsey1308/mauve/products/v3tk_v7.6.8_7000	arc:projects/mauve/products/v3tk_v7.6.8_7000

Each root contains a galaxy directory. For example, the 7000 continuum cube for NGC4698 is copied between:

```

Setonix:
/scratch/pawsey1308/mauve/products/v3tk_v7.6.8_7000/NGC4698/NGC4698
_cont_cube.fits

CANFAR transfer URI:
arc:projects/mauve/products/v3tk_v7.6.8_7000/NGC4698/NGC4698_cont_c
ube.fits

CANFAR job filesystem path:
/arc/projects/mauve/products/v3tk_v7.6.8_7000/NGC4698/NGC4698_cont_
cube.fits

```

The transfer URI is used by `vcp/vmv`. The `/arc/...` path is used by the checksum job running inside CANFAR.

The source and destination roots can be overridden through `SOURCE_NORMAL`, `SOURCE_7000`, `DEST_NORMAL` and `DEST_7000`. Automatic checksum mode currently requires `arc:` destinations; it is not a general uploader for arbitrary VOSpace endpoints.

3.2 Selected products

For each galaxy, the default selection is exactly these ten names, with `<GALID>` replaced by the directory's galaxy ID:

PRODUCT FILENAME	ROLE IN THIS TRANSFER
<code>CONFIG</code>	Preserve the configuration accompanying the products
<code><GALID>_sfh_maps.fits</code>	Transfer the SFH map product
<code><GALID>_gas_bin_maps.fits</code>	Transfer the gas-bin map product
<code><GALID>_sfh_weights.fits</code>	Transfer the SFH weights product
<code><GALID>_gas_spaxel_maps.fits</code>	Transfer the gas-spaxel map product
<code><GALID>_spatial_binning_maps.fits</code>	Transfer the spatial-binning map product
<code><GALID>_kin_maps.fits</code>	Transfer the kinematic map product
<code><GALID>_cont_cube.fits</code>	Transfer the continuum cube required by downstream work
<code>LOGFILE</code>	Preserve the nGIST processing log
<code><GALID>_mask.fits</code>	Transfer the mask product

This is a selected-product uploader, not a recursive mirror of every file in the galaxy directory. Unlisted intermediates and downstream products are not included automatically. The current source naming expects these exact, uncompressed `.fits` names; it does not discover alternate case variants or `.fits.gz` equivalents.

`--cont-only` reduces the selection to the continuum cube. Missing source continuum cubes make the galaxy fail. Other missing source products generate warnings and are skipped. Therefore, an overall success message refers to the selected **available** files; a `10/10` galaxy summary specifically confirms all ten were present and verified.

When galaxy IDs are omitted, the script discovers existing source directories from its built-in allowed galaxy list. Explicit IDs must also belong to that list. When the run argument is omitted, it selects both `normal` and `7000`.

4. Automatic checksum and repair workflow

4.1 Initial check inside CANFAR

For each galaxy, Setonix creates a small JSON request containing a unique request ID, the run, the galaxy ID and the selected filenames. It uploads that request and a small copy of the uploader script to the CANFAR run directory.

It then submits a short headless job through the CANFAR session API using the existing CADC certificate. The default job uses one core, 1 GB RAM and the image `images.canfar.net/skaha/astroml:latest`. The embedded Python reads the requested files directly through `/arc`, computes SHA256 hashes and records their byte counts.

The response is a small JSON manifest. Setonix polls for that response and checks that its request ID matches the current request. Individual lookups also validate the manifest format, run, manifest ID, filename coverage, byte count and digest format. A file that cannot be hashed is an error; it is not silently treated as missing or matching.

No manual CANFAR session needs to be activated for this workflow. It does require working certificate authentication, permission to submit CANFAR jobs and access to the destination ARC tree.

4.2 Decision for each file

Setonix computes its own SHA256 and byte count. The decision is:

CANFAR RESULT	ACTION
Same byte count and SHA256	Report verified and skip the file upload
File explicitly missing	Upload the Setonix file directly
Different byte count or SHA256	Preserve the old object under a unique quarantine name, then upload the source to the original path
Unreadable file, missing manifest coverage or invalid response	Fail the affected check rather than assume equality

For `.fits` sources, the script first checks that the first nine bytes are `SIMPLE =`. This catches an obviously invalid primary-header prefix. It is a limited input guard, not a full FITS validation or a check of HDUs, shapes, WCS, units or science values.

Before repairing a mismatch, the source is hashed again. After a successful upload, the source is hashed again to detect changes during the operation. The CANFAR hashing code also checks file metadata before and after reading. Keep product writers idle during transfer and verification: these checks are not a transactional snapshot of changing product trees.

4.3 Quarantine and replacement

An existing mismatched file is moved within CANFAR using `vmv` to a name such as:

```
NGC4698_cont_cube.fits.corrupt_<UTC timestamp>_<run ID>_<random
suffixes>
```

The old object remains available for inspection. The source is then uploaded to the original filename, which is now absent. This also avoids relying on `vcp` to overwrite an existing object that it might otherwise classify as identical.

If quarantine fails, the uploader refuses to replace the object. If the replacement upload fails after a successful quarantine, the original canonical filename may remain absent while the preserved object exists under its quarantine name. This is a repair sequence, not an atomic file swap.

The `.corrupt_*` label identifies an object replaced because it differed from the source; it does not prove how the difference arose. Quarantined objects are not automatically deleted and can consume substantial ARC storage. Review them separately after the replacement and downstream processing have been confirmed.

4.4 Fresh verification after upload

Upload success is initially recorded as awaiting verification. Once all file tasks for that galaxy finish, the uploader submits **one additional checksum job covering the newly uploaded products together**.

Only when the fresh CANFAR byte count and SHA256 match the current Setonix source does the replacement count as verified. A fresh mismatch produces an error; it does not trigger an automatic second quarantine/repair loop in that same verification step.

Normally, a galaxy needs:

- One checksum job if every selected file already matches.
- Two checksum jobs if any selected files need uploading: one initial check and one grouped post-upload check.

The jobs are deleted after their responses are received. Failed jobs, API problems or missing responses do not cause a fallback to trusting remote checksum metadata. A job submission is not automatically repeated after an uncertain POST outcome, avoiding duplicate job creation.

4.5 Persistent receipts

Before an upload attempt, the script writes a small local receipt keyed by the destination path. The receipt records the manifest ID and source hash. Reusing the same pre-upload manifest cannot initiate another repair of that file; the script reports that fresh verification is needed.

Receipts prevent an old snapshot from causing a newly uploaded replacement to be quarantined again. They are removed after matching verification. Do not remove them simply to bypass a pending-verification message; obtain a fresh manifest instead. Automatic mode does this through a new CANFAR checksum request.

5. Parallelism and bandwidth use

The current defaults are:

```
JOBS=5
TRANSFER_JOBS=10
```

These control different layers:

SETTING	MEANING	CURRENT MAXIMUM
<code>JOBS</code>	Number of galaxy workers	5
<code>TRANSFER_JOBS</code>	Shared file/CADC-operation slots across the entire invocation	10

Five galaxy workers do not restrict the run to five file uploads. Several files from one galaxy can use the shared pool simultaneously. Conversely, the defaults do not create fifty file uploads: the global pool remains capped at ten.

Each file operation holds a slot while it performs its local checking, staging, quarantine, upload and retries. Small CADC request uploads, response downloads and API operations also acquire slots. Waiting between checksum polls does not reserve a slot. Therefore, ten slots are a concurrency ceiling, not a guarantee that ten network streams are transmitting continuously.

Each file has its own staging directory and immutable manifest copy. A staging directory contains only that product beneath the original galaxy name, preventing one parallel task from removing or uploading another task's staged file. Staging uses a hard link when possible and copies the source only if linking fails.

One file is completed first for each galaxy, normally `CONFIG` when present. This establishes a missing destination directory before concurrent `vcp` calls. The initial live test exposed `DuplicateNode` errors when several calls tried to create the same absent galaxy directory; the serial first-file step addresses that race. If the first file fails, subsequent files are tried sequentially until one succeeds, after which the remaining tasks can run in parallel.

For one galaxy containing the default ten products, completing the first file leaves nine remaining file tasks. For multiple galaxies, tasks can fill the ten-slot pool from several galaxies. A single remaining continuum cube still uses one upload stream: this implementation does not split a FITS file into parallel chunks.

Galaxies are distributed round-robin into the galaxy workers' lists. Each worker advances through its own list; this is not a dynamically balanced galaxy queue. File slots are shared between all active workers.

Higher concurrency can overlap disk reads, uploads and CANFAR job waiting, but it cannot exceed available disk or network capacity. The production run demonstrates successful operation at the ten-slot setting. It is not a controlled benchmark proving that ten is faster than five or that the link's maximum bandwidth was reached.

6. CADC authentication and container overlays

6.1 Certificate reuse

At startup, the uploader checks the certificate at `~/.ssl/cadcproxy.pem` inside the CADC container environment. It checks the validity dates before contacting the transfer workflow.

CERTIFICATE CONDITION	STARTUP BEHAVIOR
Present and within its validity dates	Reuse it without calling <code>cadcc-get-cert</code>
Missing or expired	Run <code>cadcc-get-cert -u "\$CADC_USER"</code> , allowing the normal login prompt
Unreadable, invalid or not yet valid	Fail with a diagnostic instead of unexpectedly prompting

After renewal, the script checks the certificate again without prompting a second time. It does not renew periodically during a long run. Local date validation does not guarantee that every remote operation will accept the certificate; API and transfer failures remain errors requiring investigation.

The production log explicitly reports reuse of a certificate expiring on 18 October 2026 at 02:11:28 GMT. That is evidence for this run, not a fixed future expiry date.

6.2 Shared read-only overlay

The existing Setonix `vcp` wrapper runs `/cadccenv/bin/vcp` inside a Singularity container with a CADC overlay. The default base overlay is:

```
/software/projects/pawsey1308/containers/cadc_overlay.img
```

For the recognized wrapper, `CADC_READ_ONLY=auto` mounts that base overlay read-only and shares it between operations. This avoids preparing separate approximately 4 GB worker image copies. Authentication files remain in the wrapper's bound CADC home directory, outside the read-only overlay.

If a writable-overlay mode is needed, use `CADC_READ_ONLY=0`. The script then prepares a separate writable overlay for each slot under `OVERLAY_DIR`, and slot ownership prevents concurrent use of the same writable image. Those copies are removed at normal cleanup. The shared read-only base image is never removed by worker-overlay cleanup.

The script can derive temporary `vmv` and container-Python wrappers from the recognized `vcp` wrapper. These preserve its container, bindings, home directory and overlay configuration. Unsupported wrappers require appropriate explicit `VMV_CMD` or `CADC_PYTHON_CMD` settings; arbitrary wrapper formats are not automatically understood.

7. Commands for routine use

Copy the updated `.sh` to Setonix. It contains the runtime checksum helper; the local Python regression-test files do not need to be copied for normal operation.

The Setonix side needs Bash with array/`mapfile` support, GNU-style `stat` and `sha256sum`, `flock`, `pgrep`, Python 3.6+ and the CADC tools/container wrapper. The derived container-Python environment must provide the HTTP client used by the API code (`requests`). The CANFAR job image must provide Bash and Python 3, with access to the relevant `/arc` files. The script's Setonix upload mode is not intended to run unchanged on macOS.

All available 7000 galaxies

```
./vcp_scratch_v3tk_v768_to_canfar.sh 7000
```

Selected galaxies

```
./vcp_scratch_v3tk_v768_to_canfar.sh 7000 NGC4698 NGC4351
```

Only a continuum cube

```
./vcp_scratch_v3tk_v768_to_canfar.sh --cont-only 7000 NGC4698
```

Preview selections without contacting CADC

```
./vcp_scratch_v3tk_v768_to_canfar.sh --dry-run 7000 NGC4698
```

Reduce concurrency if needed

```
JOBS=2 TRANSFER_JOBS=5 ./vcp_scratch_v3tk_v768_to_canfar.sh 7000
```

To serialize both layers, use `JOBS=1 TRANSFER_JOBS=1`. Setting `JOBS=1` alone still permits parallel files within the one active galaxy. Environment values already exported in the shell override the defaults.

Capture the complete run log and exit status

```
./vcp_scratch_v3tk_v768_to_canfar.sh 7000 > uploader_7000.log 2>&1  
status=$?  
printf 'Uploader exit status: %s\n' "$status"
```

The captured log remains useful even when concurrent messages are interleaved. The per-galaxy summaries and final status are the main completion checks.

8. Settings, retries and failure handling

SETTING	DEFAULT	PURPOSE
<code>JOBS</code>	<code>5</code>	Galaxy worker count, capped at 5
<code>TRANSFER_JOBS</code>	<code>10</code>	Shared file/CADC slots, capped at 10 and reduced for small selections
<code>CADC_READ_ONLY</code>	<code>auto</code>	Read-only sharing for the recognized wrapper; <code>0</code> uses writable copies
<code>FILE_RETRIES</code>	<code>5</code>	Maximum upload attempts per file, including the first attempt
<code>RETRY_BASE_SLEEP</code>	<code>30</code>	Initial upload retry delay, seconds
<code>RETRY_MAX_SLEEP</code>	<code>240</code>	Maximum upload retry delay, seconds
<code>CHECKSUM_WAIT_SECONDS</code>	<code>1800</code>	Checksum response polling timeout, seconds
<code>CHECKSUM_POLL_SECONDS</code>	<code>10</code>	Interval between checksum response polls, seconds
<code>AUTO_CHECKSUM</code>	<code>1</code>	Automatic CANFAR checksum jobs
<code>CHECKSUM_PYTHON</code>	<code>python3</code>	Setonix embedded-Python runtime; code supports Python 3.6+
<code>CHECKSUM_MANIFEST_NAME</code>	<code>.checksum_manifest.json</code>	Static manifest filename used by manual mode
<code>CHECKSUM_RECEIPT_DIR</code>	<code>\$OVERLAY_DIR/checksum_receipts</code>	Persistent receipts for pending uploads

The default retry delays are 30, 60, 120 and 240 seconds between five attempts. Upload retries are per file. A failed file does not prevent other selected files from being processed, but the galaxy and overall run still report failure. Quarantine and initial job-submission failures are not handled by that same upload retry loop.

The checksum timeout controls the polling workflow; individual transfer/API calls have their own behavior, and waiting for a shared slot can delay an operation. It is not a strict wall-clock upper bound for every galaxy.

Ctrl-C/termination invokes descendant-process cleanup to stop active transfers before removing worker resources. Interrupted uploads are not verified successes. A rerun obtains fresh CANFAR checksums and can inspect/repair the current destination state. Temporary science staging directories may remain after an interruption; the script is not a general scratch-directory cleaner.

Exit codes

CODE	MEANING
0	Default upload run verified all selected available files; explicit no-checksum mode completed all selected available transfers with destination bytes UNVERIFIED; preview/help and no-work selections also return success without verification
1	Transfer, source, checksum, API or other processing error
2	Invalid arguments/settings, or manual-mode uploads awaiting a fresh manifest
130	Interrupted run

For normal operation, require the final verification message and the expected per-galaxy counts as well as a successful exit status. An `UPLOADED` line alone is not the final integrity result.

Optional manual fallback

Automatic mode is the normal workflow. If deliberately using a manual manifest, first run the same script inside CANFAR:

```
bash vcp_scratch_v3tk_v768_to_canfar.sh --canfar-manifest --cont-  
only 7000 NGC4698
```

Then compare from Setonix:

```
./vcp_scratch_v3tk_v768_to_canfar.sh --manual-checksum --cont-only  
7000 NGC4698
```

After manual-mode uploads, generate the manifest again inside CANFAR and rerun the Setonix command to verify them. This manual requirement does not apply to the default automatic mode.

9. Files created and retained

LOCATION	FILES/RESOURCES	LIFECYCLE
Setonix source run root	<code>.vcp_upload_stage_*</code> with per-file staging and manifest copies	Removed after a galaxy completes normally; interruption may leave staging
Setonix temporary worker directory	Slot locks, partition lists, temporary wrappers, job-ID records and error-summary records	Removed during normal exit cleanup; the summary is captured before removal and printed afterward
<code>OVERLAY_DIR</code>	Writable overlay copies when that mode is selected	Removed at cleanup; read-only sharing needs no image copies
<code>CHECKSUM_RECEIPT_DIR</code>	Destination-keyed pending receipts	Persist until successful matching verification
CANFAR run root	<code>.checksum_worker_<ID>.sh</code> , <code>.checksum_request_<ID>.json</code> , <code>.checksum_response_<ID>.json</code>	Small control artifacts retained
CANFAR galaxy directory	Replaced objects named <code>*.corrupt_*</code>	Retained for inspection; not automatically deleted
CANFAR galaxy directory in forced mode	Existing/partial destinations named <code>*.overwrite_backup_*</code>	Retained, without assuming they are corrupt; not automatically deleted
CANFAR service	Short headless checksum jobs	Deleted after valid responses; exit cleanup retries recorded job deletion

The workflow transfers science payloads from Setonix to CANFAR. In the reverse direction, it downloads only small checksum manifests, not science FITS files.

10. Production evidence from 8 October 2026

The supplied command was:

```
./vcp_scratch_v3tk_v768_to_canfar.sh 7000 \  
  NGC4216 NGC4689 NGC4351 NGC4402 NGC4405 \  
  NGC4450 NGC4457 NGC4535 NGC4698
```

The run used five galaxy workers and ten shared file slots. Its final results were:

GALAXY	INITIAL MISMATCH REQUIRING REPLACEMENT	FINAL VERIFIED COUNT
NGC4216	None recorded	10/10
NGC4689	None recorded	10/10
NGC4351	Continuum cube	10/10
NGC4402	None recorded	10/10
NGC4405	Continuum cube	10/10
NGC4450	Continuum cube	10/10
NGC4457	Continuum cube	10/10
NGC4535	Continuum cube	10/10
NGC4698	None recorded	10/10

For the five replacements, the log reports:

GALAXY	CONTINUUM SIZE, BYTES	UPLOAD DURATION REPORTED BY vcp	SPEED PRINTED BY vcp
NGC4351	1,354,584,960	26.50 s	48.75 MB/s
NGC4405	2,450,295,360	95.96 s	24.35 MB/s
NGC4457	3,710,920,320	68.97 s	51.31 MB/s
NGC4535	3,880,480,320	73.84 s	50.12 MB/s
NGC4450	3,697,387,200	119.95 s	29.40 MB/s

All five uploaded on attempt 1 of the permitted 5. The log records fourteen checksum jobs: nine initial checks and five grouped post-upload checks. Its only matched warning lines advertise a newer vos release; no failure, pending-verification or traceback lines were found in the completion review.

The 550-second total includes source hashing, CANFAR job startup and hashing, request/response transfers, quarantine operations, uploads and final verification. The individual speeds above are reproduced as printed by `vcp`; they should not be summed or treated as a measurement of the link's maximum aggregate bandwidth.

Development evidence recorded earlier in this job includes 29 passing offline regression tests and a successful live test on twenty 137-byte synthetic files across two galaxies. The synthetic test checked parallel replacement, missing-directory creation and fresh checksum verification using a shared read-only overlay. It was a functional/integrity check, not a large-file throughput benchmark. The production log supplies the subsequent real-product result.

The next downstream confirmation is to rerun the relevant mass/SFR/EWHa processing and inspect those pipeline logs. The uploader's success establishes the input-byte correspondence required for that rerun.

11. Provenance and report verification

This report describes the live local script inspected on 8 October 2026 and the user-supplied logs. It does not assume that all future remote deployments contain the same script; copy the current local version to Setonix before relying on these defaults.

Script snapshot SHA256:

```
4afde7e970383aec8e56039ce230ed7bcbdfab91a45fd6178bf4fe8664c14e2
```

Primary evidence:

1. [Current uploader script](#): source selection, configuration, certificate checks, embedded checksum job, staging, concurrency, quarantine, receipts and exit behavior.
2. [Successful nine-galaxy production log](#): nine 10/10 summaries, five replacements, fourteen job submissions and the 550-second completion.
3. [Earlier NGC4698 transfer log](#): repeated identical-file skips and failed read-back attempts in the earlier workflow.
4. [Parallel-upload regression tests](#), [checksum-repair regression tests](#) and [final-summary regression tests](#): local behavior checks retained alongside the script.
5. [8 October development job log](#): recorded development test results and the change to a ten-slot default.
6. [Forced-overwrite regression tests](#): bypass selection, backup/retry failure

handling and stale-manual-manifest protection.

The original report was checked against the source and production log without changing the uploader or rerunning transfers/pipelines. Sections 12-14 document subsequent error reporting, job diagnostics and explicit checksum bypass; the script snapshot hash above has been refreshed for these updates. The nine-galaxy production evidence remains evidence of the preceding version. The later isolated synthetic tests are described separately and do not imply a production rerun or a science-pipeline check.

12. Final error summary update

The uploader now prints a consolidated summary at the end of an unsuccessful run. Each entry identifies the run, galaxy, filename, failure stage and a short reason. For example, the following illustrates the output format rather than a new production failure:

```
FINAL ERROR SUMMARY (1 issue(s))
  run=7000 | galaxy=NGC4698 | file=NGC4698_cont_cube.fits |
stage=upload
  Upload failed after 5 attempt(s); obtain fresh checksums before
retrying.
```

The stages distinguish source discovery/header/stat/hash checks, initial manifest lookup, staging, receipt operations, quarantine, upload, checksum-job request/submission/execution/response failures and post-upload verification. If an initial checksum job fails, every requested file affected by that failed check is identified. If a post-upload job fails, the summary identifies the uploaded files whose verification remains incomplete.

Parallel tasks write separate small records in the temporary worker directory. The final summary sorts and deduplicates those records, so messages from different workers do not become interleaved summary rows. Failed upload attempts that later succeed are not recorded as outstanding errors. A successful automatic run has no error-summary block.

Manual-mode files still requiring a fresh manifest appear under a separate **AWAITING VERIFICATION SUMMARY**. This distinction is retained even when other files failed. Missing optional source files remain warnings/skips under the existing policy; they are not converted into transfer failures by this update.

For a startup/argument error before a file can be identified, the fallback summary names the available run/galaxy selection, the startup stage and exit status, and explicitly reports the file as **(not identified)**. An abrupt termination can likewise leave only a broader stage diagnostic; it does not justify inventing a specific filename or claiming verification.

The original exit status is preserved. Summary text is printed to standard error after resource cleanup, so it is visible on screen and included by the `2>&1` log-capture command in Section 7. Temporary summary records are removed; keep the captured run log if a persistent record is needed.

The update also tightens interruption ordering: parent subprocesses are stopped before their descendants are terminated, preventing an interrupted command from briefly allowing the parent shell to start another operation. Verification of this update uses local failure-injection and concurrency/repair regressions; no new science transfer or science pipeline rerun is implied.

13. Checksum-job startup failure and diagnostics

The [subsequent two-galaxy log](#) shows NGC4216 and NGC4689 timing out at the initial checksum stage after about 1807 seconds. Neither galaxy reached science-file uploads or quarantine. The twenty final-summary entries identify files whose initial verification could not be completed; they are not twenty established corruption findings.

Read-only inspection through Setonix found that those original jobs had already been deleted. A separate current checksum job was Pending with an event reporting `Back-off pulling image "images.canfar.net/skaha/astroml:latest"`. A tiny isolated test using the catalog-listed headless image `images.canfar.net/astroai/base:latest` also could not start. Its CANFAR events explicitly reported failure to reach the image registry at `206.12.94.77:443: no route to host`, followed by `ErrImagePull` and `ImagePullBackOff`. The probe job was deleted after capturing its events; no production FITS files or user-owned jobs were modified.

This fresh evidence establishes a registry-connectivity problem on the compute node used by the probe. It does not recover the unavailable historical events of the original two jobs. A different image failed with the same connectivity error, so the uploader retains its existing image default rather than claiming that an image change resolves the outage. CANFAR infrastructure must restore registry access before affected headless jobs can run; local certificate renewal, file replacement or a longer checksum timeout does not repair that network route.

The updated uploader prints each checksum job's status when it changes. While a job is Pending or Queued, it checks startup events at most once per thirty seconds. Explicit image-pull failure events end the check promptly with stage `initial-checksum/job-image-pull` or `post-upload-checksum/job-image-pull`, depending on the request phase. When reported by the event, the final reason also names the registry connectivity failure. An ordinary queue wait without those errors retains the existing timeout.

Before deleting a failed or timed-out job, the uploader now prints its events and application logs. A container that never started can have empty application logs, so the events are essential. Timeout reasons include the last observed status and image. The existing fail-closed policy remains: no checksum response means no verification claim, and a failed initial checksum request prevents science-file replacement.

After CANFAR registry connectivity is restored, copy the updated uploader to Setonix and rerun the original command. No manual checksum watcher or additional runtime helper is required. Save combined output with `2>&1 | tee` as described in Section 7 so job diagnostics remain available after cleanup.

14. Explicit forced overwrite without CANFAR checksums

Command and selection

Copy the updated self-contained script to Setonix, then run:

```
./vcp_scratch_v3tk_v768_to_canfar.sh --force-overwrite --no-checksum 7000 NGC4216 NGC4689
```

The flags also work after the run/galaxy arguments. Do not surround them with shell backticks. The command processes all selected available products for both galaxies, not just continuum cubes. Add `--cont-only` for that narrower selection. Source discovery and the required-continuum policy remain the same; optional missing products are warned/skipped. The remote run root must already exist, as in the standard upload workflow; the first successful file upload establishes a missing galaxy directory before other files run concurrently.

Preview without contacting CANFAR:

```
./vcp_scratch_v3tk_v768_to_canfar.sh --dry-run --force-overwrite --no-checksum 7000 NGC4216 NGC4689
```

Both flags are required together. Either flag alone, or combining them with `--auto-checksum/--manual-checksum`, is rejected with exit code 2. An ordinary command without these flags retains the existing default automatic checksums.

What happens for each product

1. Check the local source, including the primary FITS `SIMPLE` = card where applicable, byte count and SHA256. The local hash/change checks remain enabled; `--no-checksum` disables CANFAR checksum comparison, not these local source safeguards.
2. Read only the destination's VOSpace node definition using the existing CADC container environment. No product bytes, checksum manifest or headless session are downloaded/started. A specific missing-node response allows upload; connection/authentication/other metadata errors stop that file instead of treating it as absent. A directory at a product path is likewise an error.
3. If the destination exists, move it with `vmv` to a unique `.overwrite_backup_<timestamp>_<run-id>_<random>` name, then upload to the original path. This avoids `vcp`'s existing-file skip. The backup suffix deliberately makes no corruption claim: even a matching file is replaced in this mode.
4. On a failed upload, keep the normal per-file retry/backoff policy. Before each retry, repeat the existence check and back up any partial destination. This ensures the next attempt cannot skip the partial object left by the previous attempt.
5. Recheck the local source after a successful transfer and print `UPLOADED;` `UNVERIFIED`. Persistent receipts remain, marked as forced uploads; they are not cleared as verified.

The same five-galaxy worker limit and ten shared file/CADC slots apply. Certificate reuse, isolated staging, interruption handling and final run/galaxy/file/stage error summaries remain enabled. A failed lookup or backup move refuses the replacement upload and is included in the final summary. If upload fails after the original destination was renamed, the original bytes remain under their backup name, while the original path may be absent or partial; inspect these before downstream processing or another retry. Backup files accumulate until deliberately reviewed/removed.

Completion and later verification

Successful transfers return exit code **0**, with a final message stating that destination bytes are **UNVERIFIED** and CANFAR checksum verification was skipped. This is transfer completion, not an integrity result. Errors still return 1 and preserve the existing final error summary. The intentional no-checksum mode does not queue a post-upload job or return manual-mode pending status merely because verification was skipped.

After CANFAR checksum jobs can run again, remove both bypass flags and obtain fresh verification:

```
./vcp_scratch_v3tk_v768_to_canfar.sh 7000 NGC4216 NGC4689
```

The normal automatic request checks actual CANFAR bytes and clears a forced receipt only after a matching response. A forced receipt blocks manual-mode verification, even if a static manifest reports matching bytes, because that snapshot may predate the forced upload. Use the default automatic mode to verify forced uploads; do not delete receipts or treat an old manual snapshot as fresh evidence.

Validation evidence and limits

A live test ran through Setonix into an isolated ARC home directory using two synthetic galaxies, each with a CONFIG and a continuum-named 93-byte payload. After preparing the remote run-root prerequisite, the first run uploaded four missing files. The second run replaced all four after modifying their source bytes. A separate test-only read-back of those tiny synthetic objects confirmed that the four new payloads matched their sources and all four backups retained the original bytes. No production science products or user-owned jobs were changed, and the forced uploader launched no CANFAR checksum jobs. Tiny synthetic read-back was a test check, not functionality added to the uploader.

Offline regressions also exercise missing destinations, metadata and backup failures, partial-upload retry, flag validation, usage placement and stale-manual-manifest protection alongside the existing integrity/concurrency/error-reporting tests. This establishes functionality, not large-file throughput or verified integrity for a future production run with checksums disabled.

The complete offline suite passed 52 tests; targeted forced-mode checks passed 11 tests again after the final help wording and summary assertions were updated. Bash syntax and embedded Python compatibility were checked separately.